Antes de la sesión 9: checklist de preparación

1MTR68 · Prototipado y Manufactura de Productos Mecatrónicos

⏱️ **Tiempo estimado: 30 minutos.** Hazlo **antes** de la clase. Si llegas sin esto, pierdes la primera media hora de la sesión instalando.

Al final de esta guía hay una **prueba final**. Si la pasa, estás listo.

---

1. Crea tu cuenta de GitHub

1. Entra a https://github.com/signup y crea una cuenta. Puedes usar tu correo PUCP.
2. Elige un nombre de usuario profesional (lo verán futuros empleadores): `jperez-mecatronica` mejor que `xXgamer99Xx`.
3. Verifica tu correo.

2. Instala VS Code y KiCad 10

Instálalos **antes que Git**: el instalador de Git te preguntará qué editor usar y VS Code ya debe estar en tu PC.

- **Visual Studio Code** (para editar el README y los mensajes de Git): https://code.visualstudio.com/. En Windows, deja marcada la opción **"Agregar a PATH"** (*Add to PATH*), que viene marcada por defecto.
- **KiCad 10** (la última versión 10.0.x): https://www.kicad.org/download/. Si tienes KiCad 9 instalado, puedes conservarlo: las versiones conviven sin problema.

3. Instala Git

Windows

Descarga **Git for Windows** de https://git-scm.com/download/win. El instalador tiene muchas pantallas: en casi todas basta con **Next**, pero **revisa estas tres**:

Pantalla del instalador	Qué elegir	Por qué
Select Components	Deja marcado *Windows Explorer integration → Open Git Bash here* y **marca** *Add a Git Bash Profile to Windows Terminal*	Para abrir Git Bash con clic derecho o desde la app Terminal
Choosing the default editor used by Git	**Use Visual Studio Code as Git's default editor**	El editor por defecto es Vim: si Git lo abre, es muy difícil salir
Adjusting the name of the initial branch in new repositories	**Override the default branch name for new repositories** → `main`	Así tu rama principal se llama igual que en GitHub

En el resto deja la opción por defecto. En especial, **no cambies** *Adjusting your PATH environment* (deja la opción *Recommended*) ni *Choose a credential helper* (deja **Git Credential Manager**: es lo que te permite iniciar sesión en GitHub desde el navegador).

> ¿Ya tenías Git instalado? Puedes volver a ejecutar el instalador encima para cambiar estas opciones, o simplemente hacer el paso 5 (configura lo mismo con comandos).

macOS

Abre la app **Terminal** y escribe `git --version`. Si no está instalado, macOS te ofrecerá instalar las *Command Line Developer Tools*: acepta.

Linux

`sudo apt install git` (Ubuntu/Debian) o el equivalente de tu distribución.

¿Qué terminal uso?

- **Windows:** recomendamos **Git Bash** (los comandos son iguales a los de macOS/Linux). Los comandos de las guías **también funcionan en PowerShell**, así que si abriste PowerShell sin querer no pasa nada.
- **macOS / Linux:** la app **Terminal**.

> **Abrir Git Bash en una carpeta (Windows):** clic derecho dentro de la carpeta → *Open Git Bash here* (en Windows 11: *Mostrar más opciones* → *Open Git Bash here*).
>
> **¿Usas clic derecho → *Abrir en Terminal*?** En Windows 11 eso abre PowerShell. Para que abra Git Bash: abre *Terminal* → flecha ⌄ junto a la pestaña → **Configuración** → **Inicio** → **Perfil predeterminado** → **Git Bash** → **Guardar**. Si Git Bash no aparece en la lista, vuelve a ejecutar el instalador de Git y marca *Add a Git Bash Profile to Windows Terminal*.
>
> **¿Cómo sé en cuál estoy?** PowerShell muestra `PS C:\...>`. Git Bash muestra `usuario@PC MINGW64 ~` y debajo un `$`.

4. Configura la autenticación con GitHub

GitHub ya no acepta tu contraseña desde la terminal. Hay que configurar un método de acceso:

- **Windows:** no hagas nada. Git for Windows trae el *Git Credential Manager*: la primera vez que hagas `git push` se abrirá el navegador para que inicies sesión en GitHub.
- **macOS / Linux:** instala la herramienta de GitHub (`gh`) y úsala para autenticarte:

```
# macOS (necesitas Homebrew: https://brew.sh)
brew install gh
# Ubuntu/Debian: ver https://github.com/cli/cli/blob/trunk/docs/install_linux.md

gh auth login
```

Responde: **GitHub.com** → **HTTPS** → *Authenticate Git with your GitHub credentials?* **Yes** → **Login with a web browser**.

5. Configura Git (una sola vez por computadora)

Abre la terminal y copia **los cuatro** comandos, **cambiando nombre y correo**. Hazlo aunque ya hayas elegido VS Code y `main` en el instalador: si tenías Git instalado de antes, puede haber quedado otra configuración.

```
git config --global user.name "Juan Pérez"
git config --global user.email "tu-correo@pucp.edu.pe"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

Comando	Para qué sirve
`user.name` / `user.email`	Firma de tus commits. Usa el **mismo correo** de tu cuenta GitHub.
`init.defaultBranch main`	La rama principal se llamará `main`, igual que en GitHub.
`core.editor "code --wait"`	Si Git necesita que escribas un mensaje, abrirá VS Code (y no *vim*, del que es difícil salir).

Verifica preguntándole a Git cada valor:

```
git config user.name
git config user.email
git config init.defaultBranch
git config core.editor
```

🔍 **Debe responder:** tu nombre, tu correo, `main`, y `code --wait` (o la ruta de VS Code, por ejemplo `"C:\...\Code.exe" --wait`, si lo elegiste en el instalador).

⚠️ No uses `git config --global --list` para verificar: en Windows, lo que eliges en el instalador se guarda en la configuración **del sistema**, así que esa lista no lo muestra. Si `init.defaultBranch` responde `master`, vuelve a ejecutar `git config --global init.defaultBranch main`.

6. Elige tu carpeta de trabajo

Todo lo de la sesión vivirá en **una carpeta de trabajo**: ahí estarán el material del curso (`1MTR68/`) y tu proyecto (`demogit/`), uno al lado del otro. Elige **una** de estas opciones y úsala siempre:

Opción A (recomendada): la carpeta `proyectos` en tu usuario. Abre la terminal en cualquier lugar y escribe:

```
mkdir -p ~/proyectos
cd ~/proyectos
```

`~` significa **tu carpeta de usuario** (en Windows, `C:\Users\TU_USUARIO`; en macOS, `/Users/TU_USUARIO`). Estos comandos te llevan a `~/proyectos` **aunque hayas abierto la terminal en otra carpeta**.

Opción B: una carpeta que tú ya tienes (por ejemplo `D:\Proyectos`). Ábrela en el Explorador, clic derecho → *Open Git Bash here* (o *Abrir en Terminal*). **No** escribas los comandos de la opción A: ya estás donde debes.

> Para ir a esa carpeta escribiendo: en PowerShell `cd D:\Proyectos`; en Git Bash las unidades se escriben distinto: `cd /d/Proyectos`.

Sea cual sea tu opción, comprueba dónde estás:

```
pwd
```

🔍 Debe mostrar tu carpeta de trabajo (por ejemplo `/c/Users/jperez/proyectos` en Git Bash, o `C:\Users\jperez\proyectos` en PowerShell). **Anótala**: en clase empezarás siempre desde ahí.

7. ✅ Prueba final

Desde tu carpeta de trabajo:

```
git --version
git clone https://github.com/hector310595/GitForHardwareClass.git 1MTR68
ls 1MTR68
```

Debes ver algo como:

```
git version 2.4x.x
Cloning into '1MTR68'...
README.md  guia  inicio  recursos
```

Si llegaste aquí: **estás listo para la sesión.** 🎉

> ⚠️ **Tu carpeta de trabajo no debe estar dentro de OneDrive, Google Drive ni Dropbox.**
> Sincronizar una carpeta de Git con la nube puede corromper el repositorio. Ojo: en muchas PCs con Windows, *Documentos* y *Escritorio* están dentro de OneDrive.

¿Problemas?

Síntoma	Solución
`git: command not found` / *"git no se reconoce"*	Cierra y vuelve a abrir la terminal: las terminales abiertas antes de instalar Git no lo ven.
`1MTR68` se creó en otra carpeta (clonaste desde un lugar distinto a tu carpeta de trabajo)	Abre la terminal en tu carpeta de trabajo, comprueba con `pwd` y vuelve a clonar ahí (paso 7). Luego borra la copia equivocada: en PowerShell `Remove-Item -Recurse -Force RUTA\1MTR68`, en Git Bash `rm -rf RUTA/1MTR68`.
`code: command not found`	En VS Code: `Ctrl/Cmd+Shift+P` → *Shell Command: Install 'code' command in PATH*. En Windows reinstala VS Code marcando "Add to PATH".
No encuentro mi carpeta en el Explorador	Escribe `explorer .` (con el punto): abre en el Explorador la carpeta donde estás.
Al hacer commit se abrió una pantalla negra con `~` a la izquierda (Vim)	No elegiste VS Code como editor
`gh auth login` falla	Usa la opción de pegar un *token*: GitHub → Settings → Developer settings → Personal access tokens.

Si nada funciona, escribe al docente **antes** de la clase con una captura del error.